

Contexto del Negocio

Eres parte del equipo de BI de una empresa de e-commerce. Se te proporciona acceso a un proyecto en BigQuery con los siguientes datos crudos en el esquema `raw`:

- `raw.orders` — pedidos realizados
- `raw.order_items` — ítems por pedido
- `raw.customers` — clientes
- `raw.products` — catálogo de productos

Los datos tienen problemas de calidad: campos nulos, fechas mal formateadas, IDs duplicados y registros inconsistentes. Tu misión es construir un pipeline de transformación con dbt que entregue datos confiables y listos para análisis.

Entregables

Se evaluarán **4 secciones** con los siguientes pesos:

Sección	Tema	Puntos
1	Modelado con dbt	40
2	Calidad de datos & tests	25
3	Orquestación con Airflow	20
4	Análisis & documentación	15
Total		100

Sección 1 – Modelado con dbt (40 pts)

1.1 Estructura de capas (10 pts)

Crea un proyecto dbt con la siguiente estructura de capas:

```
None
models/
  staging/      ← limpieza 1:1 de tablas raw
  intermediate/ ← lógica de negocio
```

`marts/`

← tablas finales para consumo

Se espera al menos:

- 4 modelos `staging` (uno por tabla raw)
- 2 modelos `intermediate`
- 2 modelos en `marts`

1.2 Modelos requeridos (20 pts)

Construye los siguientes modelos en la capa `marts`:

`mart_orders_summary` — Una fila por orden, con:

- `order_id`, `customer_id`, `order_date`
- `total_items` (cantidad de ítems)
- `total_revenue` (suma del precio × cantidad)
- `order_status` normalizado (en minúsculas, sin espacios)

`mart_customer_ltv` — Una fila por cliente, con:

- `customer_id`, `customer_name`, `email`
- `first_order_date`, `last_order_date`
- `total_orders`, `total_revenue`
- `customer_segment`: 'VIP' si revenue > \$5,000, 'Regular' si revenue > \$500, 'Nuevo' si tiene menos de 2 órdenes

1.3 Macros y reutilización (10 pts)

Crea al menos **una macro dbt** reutilizable. Sugerencias:

- Macro para normalizar texto (`trim`, `lower`)
- Macro para calcular segmento de cliente
- Macro para aplicar un filtro de fechas estándar

Úsala en al menos dos modelos.

Sección 2 – Calidad de datos & Tests (25 pts)

2.1 Tests genéricos (10 pts)

En el archivo `schema.yml`, aplica tests dbt a los modelos `staging` y `marts`:

- `not_null` en columnas críticas
- `unique` en PKs
- `accepted_values` donde corresponda (ej. `order_status`)
- `relationships` entre tablas (foreign keys)

Se espera **mínimo 10 tests** en total.

2.2 Test singular personalizado (10 pts)

Crea un test singular en `tests/` que valide la siguiente regla de negocio:

"Ninguna orden puede tener un `total_revenue` negativo o igual a cero."

El test debe fallar si encuentra registros que violen esta regla.

2.3 Source freshness (5 pts)

Configura `sources.yml` con las tablas raw y agrega configuración de `freshness` para alertar si los datos de `raw.orders` tienen más de **24 horas** sin actualizarse.

Sección 3 – Orquestación con Airflow (20 pts)

3.1 DAG básico (15 pts)

Crea un DAG en Airflow que:

1. Corra a las **2:00 AM** todos los días
2. Ejecute `dbt run --select staging` primero
3. Luego ejecute `dbt test --select staging`
4. Si los tests pasan, ejecute `dbt run --select marts`
5. Finalmente ejecute `dbt test --select marts`

Puedes usar el operador `BashOperator` o `DbtRunOperator` del paquete `astronomer-cosmos`.

El DAG debe tener manejo de errores: si `dbt test` falla, el pipeline debe detenerse y no continuar al siguiente paso.

3.2 Pregunta teórica (5 pts)

Responde en máximo 150 palabras:

¿Qué estrategia usarías para manejar tablas grandes en BigQuery (más de 500 millones de filas) dentro de tus modelos dbt? Menciona al menos dos conceptos específicos de dbt o BigQuery que aplicarías.

Sección 4 – Análisis & Documentación (15 pts)

4.1 Documentación dbt (5 pts)

En el `schema.yml`, agrega:

- `description` en todos los modelos de `marts`
- `description` en al menos 5 columnas de `mart_customer_ltv`

Ejecuta `dbt docs generate` y adjunta captura del lineage graph.

4.2 Consulta analítica en BigQuery (10 pts)

Escribe una query SQL directamente en BigQuery (sin dbt) que responda:

"¿Cuáles son los 5 productos más vendidos por mes durante el último trimestre, y cuál fue su variación porcentual en revenue vs el mes anterior?"

La query debe usar **window functions** (`LAG`, `RANK` o similar).

Datos de muestra

Se proporcionan los siguientes scripts para poblar las tablas raw en BigQuery:

```
SQL
-- raw.orders (incluye intencionalmente datos sucios)
CREATE TABLE raw.orders AS
SELECT * FROM UNNEST([
  STRUCT('0001' AS order_id, 'C01' AS customer_id,
    '2024-01-15' AS order_date, 'completed' AS status),
  STRUCT('0002', 'C02', '2024-01-16', 'COMPLETED'),
```

```

    STRUCT('0002', 'C02', '2024-01-16', 'COMPLETED'), --
duplicado intencional
    STRUCT('0003', 'C03', NULL, 'pending'),
    STRUCT('0004', 'C01', '2024-02-01', 'cancelled'),
    STRUCT('0005', 'C99', '2024-02-10', 'completed') -- cliente
inexistente
]);

-- raw.customers
CREATE TABLE raw.customers AS
SELECT * FROM UNNEST([
    STRUCT('C01' AS customer_id, 'Ana García' AS name,
'ana@email.com' AS email),
    STRUCT('C02', 'Carlos López', 'carlos@email.com'),
    STRUCT('C03', NULL, 'sin_nombre@email.com')          -- nombre
nulo intencional
]);

-- raw.products
CREATE TABLE raw.products AS
SELECT * FROM UNNEST([
    STRUCT('P01' AS product_id, 'Laptop' AS name, 1200.00 AS
price),
    STRUCT('P02', 'Mouse', 25.00),
    STRUCT('P03', 'Teclado', 75.00)
]);

-- raw.order_items
CREATE TABLE raw.order_items AS
SELECT * FROM UNNEST([
    STRUCT('0001' AS order_id, 'P01' AS product_id, 1 AS
quantity),
    STRUCT('0001', 'P02', 2),
    STRUCT('0002', 'P03', 3),
    STRUCT('0004', 'P99', 1), -- producto inexistente
    STRUCT('0005', 'P01', -1) -- cantidad negativa intencional
]);

```

Entrega

El candidato debe compartir:

1. Repositorio Git con el proyecto dbt completo
2. Archivo del DAG de Airflow (`.py`)
3. Captura del lineage graph de dbt docs
4. La query SQL del punto 4.2
5. Un `README.md` con instrucciones para correr el proyecto